

Legal Metrology — Member 3 Status Report

Flutter Android App (Inspector + Business) · Smart India Hackathon 2026

DELIVERABLE

Flutter APK — Inspector + Business/Retailer roles, fully working in Demo Mode (mock backend).

STATUS

100% of the Flutter app build is complete — analyze 0 errors, 8/8 tests passing, debug APK builds. Remaining work is **integration**, not app development.

Location: **E:\SIH_2026\Legal_Metrology\mobile_app** (inside the team GitHub repo, own .gitignore so build artifacts stay out of team commits). Stack: Flutter 3.44 / Dart 3.12, Riverpod, go_router, Dio, flutter_secure_storage, camera + image_picker, signature pad. Material 3 light GovTech theme.

1. Completed — App Foundation

- Complete project scaffold: pubspec, Android config (minSdk 23, compileSdk 36, camera permissions), Material 3 light GovTech theme.
- Clean architecture: UI → controllers → repository interfaces → Mock/Real implementations. UI never imports mock data or Dio directly.
- 13 domain models mapped to the shared PostgreSQL entities (users, businesses, products, inspections, evidence, ocr_results, violations, notices, cases, payments, supply chain...).
- 11 repository interfaces = the API contract for Member 1 (auth, business, inspection, evidence, OCR, violation, offence, notice, seizure+case, self-check, payment, supply chain).
- Typed error layer — every failure surfaces a friendly message with Retry (401/403/404/timeout/offline/upload/OCR/notice/payment). No stack traces ever reach the UI.
- Auth: secure token storage (Keystore), JWT bearer interceptor with single-attempt refresh on 401, session restore at splash, role-based route guards + unauthorized screen. Inspector self-registration is impossible by design.

2. Completed — Inspector App (Enforcement Flow)

- **Dashboard** — assigned inspections, active cases, draft notices, pending actions, quick actions, today's inspections.
- **Business search** — name/GSTIN/owner/city; Start Inspection sheet (Routine / Complaint-Based / Supply-Chain-Linked + complaint ID).
- **8-step guided inspection flow:** multi-angle evidence capture (front/back/side slots, retake/delete/preview) → 5-stage OCR pipeline UI with retry → fully editable OCR review (add/remove/correct fields, confidence chips) → AI violation review (Accept/Reject/Edit + manual Add, POTENTIAL→confirmed human-in-the-loop) → previous product offence history (tier badge, case records) → observations + supplier declaration + seizure/sample with witnesses → notice generation & review (AI-DRAFT banner, editable body, add sections from Legal-KB library) → signature pad → issue.
- **Case timeline** — visual done/current/pending states with actors and details; only backend-returned states rendered.

3. Completed — Business App (Prevention Flow)

- **Dashboard** — PREVENTIVE COMPLIANCE hero ("Check your packaging before you sell"), stats, quick actions, private-data banner.
- **Private self-check** — capture package photos, pipeline progress, COMPLIANT / POTENTIAL ISSUES result with plain-language requirement + recommended correction per issue. Clearly labelled PRIVATE throughout.
- **Self-check history** — private reports with result chips.
- **Notice inbox + detail** — status-gated actions: Submit Correction (photos + comments), Raise Dispute, Give Consent (explicit confirm), Pay Penalty.
- **Case tracking** — Inspection → Notice → Response → Correction → Compounding → Payment → Closure timeline.

- **Payments** — pending + history, ■ en_IN formatting, backend-verified status polling (app never marks success locally).

4. Completed — Demo Mode & Quality

- Stateful MockBackend simulating NestJS + FastAPI: realistic Indian seed data, RBAC, latency, staged OCR with small failure rate, notice/payment status lifecycles — the full judge demo runs with zero backend.
- Demo Mode ON/OFF via one flag or dart-defines; per-capability migration (REAL_AUTH / REAL_OCR) supported.
- Demo credentials: inspector rajesh.deshmukh / inspector123 · business anita@abctraders.in / business123.
- Reusable widget library (25+ components), loading/error/empty states everywhere, 8/8 automated tests passing, flutter analyze 0 errors/0 warnings.

5. Remaining (Integration-Phase Work)

Task	Blocked on	Owner
Swap Mock Auth → Keycloak OIDC via NestJS	Keycloak config + final auth endpoints	Member 1
Swap all Mock repos → real NestJS API routes + JSON casing	Final REST contract	Member 1
Point OCR flow at FastAPI passthrough (job polling)	OCR response schema freeze	Members 1 + 4
Wire Legal-KB section library API (replace 8 mock sections)	Sections API / rule mappings	Member 5
eMudhra signature implementation behind SignatureService	DSC/eSign integration spec	Member 6
Razorpay checkout sheet + order/webhook flow	Payment initiate response shape	Members 1 + 6
file_picker for document attachments (invoices/dispute docs)	Live backend for uploads	Member 3 (me)
Offline capture queue (future scope, architecture already allows)	Post-integration	Member 3 (me)

6. How to Integrate With Each Teammate

MEMBER 1 — NestJS + PostgreSQL (Integration Owner)

They give me: final REST endpoints + JSON schemas, Keycloak realm config, role claims in JWT, multipart upload contract, ID generation server-side.

I give them: the repo file `mobile_app/lib/repositories/*` — these interfaces ARE the contract (11 capability groups). Also `mobile_app/DEVELOPMENT_REPORT.md` §G lists every expected route.

Integration steps: (1) share endpoint list → I update only `mobile_app/lib/data/real/real_repositories.dart`; (2) they stand up a dev server → I run the app with `--dart-define=API_BASE_URL=http://<host>:3000/api/v1`; (3) migrate capability-by-capability: `--dart-define=REAL_AUTH=true` first, then the rest; (4) joint test of 401/403 paths + token refresh.

Rule: Flutter NEVER talks to PostgreSQL directly — all data via their API.

MEMBER 2 — Next.js Web (Controller + Citizen)

No direct integration — our apps meet only inside PostgreSQL via NestJS.

Shared-state points to verify together in a joint demo: Controller assigns inspection → appears in my Inspector app; Citizen complaint → complaint-based inspection appears with complaint ID; I issue notice → it must render on their Controller console; business pays → closure shows on Controller web.

Agreement needed: common case/inspection/notice ID formats so every screen shows the same identifiers (my mocks already use INSP-2026-xxxxx / LM/2026/xxxx / NOT-xxxxxx — confirm or replace).

MEMBER 4 — FastAPI AI (OCR / NER / Compliance / NLP)

They give me (through Member 1's passthrough, never directly): OCR job envelope (jobId, status, progressStep, failureReason), fields[] with keys PRODUCT_NAME, GENERIC_NAME, MANUFACTURER, PACKER, IMPORTER, NET_QUANTITY, MRP, MANUFACTURING_DATE, EXPIRY_USE_BY, COUNTRY_OF_ORIGIN, CONSUMER_CARE, BATCH_LOT, OTHER — each with value + confidence (0-1) + isMissing; violations[] (type, description, severity, status=POTENTIAL, confidence, ruleSection, ruleTitle, recommendation); self-check issues in plain business language; offence lookup (matchedProductName, tier, matchConfidence, records[]).

I give them: multi-image JPEG uploads (front/back/side), editable-field confirmations so corrections can feed their feedback loop, human-in-the-loop guarantee (AI never finalises — their POTENTIAL status is preserved in UI).

Full field spec: mobile_app/DEVELOPMENT_REPORT.md §H.

MEMBER 5 — Legal Knowledge Base

They give me: sections as {id, citation, title, description} — e.g. "Rule 6(3), LM (Packaged Commodities) Rules, 2011"; violation→section mappings; compounding amounts by offence tier; notice-period defaults for panchanama/seizure.

I give them: the NoticeSection model my UI consumes (mock ships 8 representative sections in mobile_app/lib/data/mock_data.dart — their API output should match that shape).

Used at: Add-Violation section picker, notice draft sections + Add Section, and self-check REQUIREMENT text.

MEMBER 6 — Razorpay / eMudhra / WhatsApp / Email / GST

Razorpay: my initiatePayment call must receive {paymentId, orderId, amount, currency, note} to open checkout; status polled via getPaymentStatus; webhook on their side is the ONLY success source — my UI explicitly never marks success locally. I need their checkout key/flow decision (SDK vs backend-hosted page).

eMudhra: implement my SignatureService interface (mobile_app/lib/services/signature_service.dart) with DSC/eSign; verifyDigitalSignature() already exists as the hook. Drawn-signature prototype is clearly labelled as not legally valid.

WhatsApp/Email: no app work needed — notice delivery notifications fire server-side after I POST the signed notice.

GSTIN verification: registration flow already routes through backend; I display pending-verification status they set.

7. Proposed Joint Timeline (Next 2 Weeks)

- **Week 1:** Member 1 finalises auth + 3 core endpoints (businesses search, inspections, evidence upload) → I integrate real auth and those flows; joint smoke test on a real device.
- **Week 1:** Member 4 freezes OCR JSON schema → I wire REAL_OCR and test with real package photos.
- **Week 2:** notices/violations/self-check endpoints; Member 6 payment initiate response → Razorpay checkout wired; end-to-end demo rehearsal: complaint → inspection → OCR → confirm → notice → business response → compounding → payment → closure across all four apps.

Hand-off docs in the repo: mobile_app/README.md (run + demo-mode guide) · mobile_app/DEVELOPMENT_REPORT.md (full integration report §G–N). App verified: flutter analyze 0 errors · 8/8 tests · debug APK builds.